

MINUTES OF MEETING (MOM)

Team 8

3D Printing Cost Estimation Platform

Meeting Details

Project: 3D Printing Cost Estimation Platform
Week: Week 4
Date: Thursday, February 13, 2026
Time: 4:00 PM IST
Location: CIE, Ground Floor
Client: Anil Kumar Upadhyay, Five Fingers Innovative Solutions
Client Email: aku1974india@gmail.com
Client Phone: 9949497560

Team Members Present

- Abhiroop Pullepu
- Mehrish Khan
- Ronith Menneni
- Sai Hasini
- Ruschita Guddeti

Notes

- Sprint 1 review and demo conducted
- File upload API successfully demonstrated
- Client provided feedback on next phase priorities
- Release 1 demo deadline discussed and finalized

1 Sprint 1 Review & Demo

1.1 Completed Features

- [3DP-1] File Upload API:
 - POST /api/upload endpoint functional
 - STL/STEP file validation working
 - 50MB file size limit enforced
 - UUID job.id generation implemented
 - Files saved to /uploads/{job.id}/model.ext structure
 - Database records created successfully
 - Swagger documentation complete

- **[3DP-2] Database Schema:**
 - PostgreSQL database setup complete
 - jobs, materials, failure_logs tables created
 - SQLAlchemy ORM models implemented
 - Database migrations working (Alembic)
- **[3DP-3] CI/CD Pipeline:**
 - GitHub Actions workflows active
 - Backend CI: lint, format check, pytest (5/5 passing)
 - Branch protection rules enforced on main/dev
 - Docker Compose build verification passing

1.2 Demo Highlights

1. Swagger UI shown at localhost:8000/docs
2. Live upload of sample STL file demonstrated
3. Job_id returned and verified in database
4. File saved correctly in directory structure
5. Error handling demonstrated (wrong file type, oversized file)
6. Unit tests executed live (all passing)

1.3 Client Feedback

- **Positive:** Clean API design, good documentation, solid error handling
- **Request:** Prioritize CURA slicing integration in Sprint 2
- **Request:** Ensure G-code preview is interactive (layer-by-layer view)
- **Concern:** Need to see 3D viewer working soon to validate mesh rendering approach

2 Sprint 2 Planning

2.1 Priority Features

1. **[3DP-10] CURA Slicing Engine Integration (High Priority)**
 - Setup CuraEngine in Docker container
 - Implement POST /api/slice/{job_id} endpoint
 - Parse G-code for print time and material usage
 - Store slicing results in database
 - Handle slicing failures with detailed error logs
2. **[3DP-15] Basic 3D Model Viewer (High Priority)**
 - Three.js scene setup with STLLoader
 - OrbitControls for rotation/zoom/pan
 - Display model dimensions
 - Basic lighting and camera positioning
 - Grid helper (build plate visualization)

3. [3DP-20] Slicing Parameters Form (Medium Priority)

- UI form for material selection, layer height, infill
- Frontend validation
- POST parameters to slicing endpoint
- Display slicing progress indicator

4. [3DP-25] Cost Calculation Engine (Medium Priority)

- Implement cost formula from SRS
- Material cost + waste factor + failure factor
- Machine time cost calculation
- POST /api/estimate/{job_id} endpoint
- Cost breakdown response schema

2.2 Story Point Allocation

- **Total Sprint Capacity:** 40 story points (8 points/person $\times$ 5 members)
- **Sprint 2 Commitment:** 35 points (buffer for unknowns)
 - CURA Integration: 13 points
 - 3D Viewer: 8 points
 - Slicing Form: 5 points
 - Cost Engine: 8 points
 - Documentation: 1 point

3 Release 1 Timeline & Scope

3.1 Release 1 Demo Deadline

- **Date:** Friday, February 28, 2026 (end of Sprint 2)
- **Time:** 4:00 PM IST
- **Location:** CIE Ground Floor (with projector for live demo)

3.2 Release 1 Scope (MVP Features)

1. File upload with validation
2. STL model visualization (Three.js viewer)
3. CURA slicing with configurable parameters
4. G-code preview (basic layer view)
5. Cost estimation with breakdown
6. Admin material configuration (basic CRUD)
7. Simple authentication (username/password)

3.3 Release 1 Acceptance Criteria

- All Sprint 1 + Sprint 2 features working end-to-end
- Docker Compose brings up entire stack (frontend + backend + DB)
- Complete upload → slice → visualize → estimate workflow
- No critical bugs (P0/P1 issues resolved)
- 80%+ test coverage on backend
- All documentation up to date (README, API docs, diagrams)

4 GitHub & Jira Best Practices (Reinforced)

4.1 Code Quality Requirements

- **All code must pass CI before merge:**
 - Linting (pylint, ESLint)
 - Formatting (black, Prettier)
 - Type checking (mypy, TypeScript strict)
 - Unit tests (pytest, Vitest)
- **Pull Request Standards:**
 - PR title format: “[3DP-XX] Short description”
 - Description includes: What changed, why, testing done
 - Screenshots for UI changes
 - Link to Jira issue
 - Request review from at least one team member
- **Code Review Checklist:**
 - Does it solve the issue completely?
 - Are there tests for new functionality?
 - Is code readable and well-documented?
 - Are there any security concerns?
 - Does it follow project conventions?

4.2 Branch Management

- **Never commit directly to main or dev**
- **Feature branch workflow:**
 1. `git checkout dev`
 2. `git pull origin dev`
 3. `git checkout -b feature/3DP-XX-short-name`
 4. Make changes, commit frequently
 5. `git push origin feature/3DP-XX-short-name`
 6. Create PR on GitHub
 7. After approval + CI pass, merge to dev
 8. Delete feature branch
- **Keep branches short-lived:** Merge within 2-3 days to avoid conflicts
- **Resolve merge conflicts locally before pushing**

4.3 Jira Issue Management

- **Update issue status promptly:**
 - Move to “In Progress” when you start work
 - Move to “Code Review” when PR created
 - Move to “Testing” after merge to dev
 - Move to “Done” after QA validation
- **Log work hours:** Use Jira time tracking for accurate burndown
- **Comment on blockers immediately:** Don’t wait for standup
- **Link related issues:** “Blocked by”, “Relates to”, “Duplicates”

5 Testing & Quality Assurance

5.1 Testing Pyramid

- **Unit Tests (Base):** 70% of tests
 - Test individual functions in isolation
 - Mock external dependencies (database, APIs)
 - Fast execution (< 5 seconds total)
- **Integration Tests (Middle):** 20% of tests
 - Test API endpoints with real database
 - Verify data flows correctly through layers
 - Use test database (not production)
- **E2E Tests (Top):** 10% of tests
 - Test complete user workflows
 - Use Playwright/Cypress for frontend
 - Run before releases only (time-consuming)

5.2 Test Coverage Goals

- **Backend:** 80%+ code coverage (enforce in CI)
- **Frontend:** 70%+ component coverage
- **Critical paths:** 100% coverage (upload, slice, estimate)

5.3 Bug Reporting Process

1. Create Jira issue with type “Bug”
2. Priority: P0 (critical), P1 (high), P2 (medium), P3 (low)
3. Include:
 - Steps to reproduce
 - Expected vs actual behavior
 - Environment (OS, browser, Docker version)
 - Screenshots or logs

4. Assign to appropriate developer
5. Link to user story that introduced bug
6. **P0/P1 bugs block release**

6 Technical Discussions

6.1 CURA Engine Docker Setup

- **Base Image:** ubuntu:22.04
- **Installation:**
 1. Download CuraEngine binary from Ultimaker GitHub releases
 2. Install dependencies: libstdc++6, libgomp1
 3. Copy fdmprinter.def.json (printer definition file)
 4. Set up volume mounts for /uploads and /gcode
- **Execution Strategy:**
 - Backend calls CuraEngine via subprocess
 - Command format: `CuraEngine slice -j fdmprinter.def.json -l model.stl -o output.gcode -s layer_height=0.2 ...`
 - Timeout after 120 seconds (log failure)
 - Parse G-code output for time/material data
- **Error Handling:**
 - Capture stderr for error messages
 - Log failures to failure_logs table
 - Return user-friendly error messages (e.g., “Model has thin walls ; 0.8mm”)

6.2 Three.js Performance Optimization

- **Use BufferGeometry** (not deprecated Geometry)
- **Implement LOD** (Level of Detail) for large models:
 - High detail when zoomed in
 - Low detail (decimated mesh) when zoomed out
- **Web Workers:** Parse large STL files off main thread
- **Dispose resources:** Call dispose() on geometries/materials on unmount
- **Throttle render:** Only re-render on camera movement, not every frame

6.3 Database Connection Pooling

- SQLAlchemy pool size: 10 connections
- Max overflow: 5 additional connections
- Connection timeout: 30 seconds
- Pool recycle: 3600 seconds (1 hour)
- Use get_db() dependency in FastAPI for automatic session management

7 Documentation Requirements

7.1 Control Flow Diagram

- **Tool:** Mermaid.js
- **Location:** /docs/diagrams/control-flow.md
- **Content:**
 - High-level system flow
 - User uploads file → Validation → Storage → Database
 - User configures params → Slicing → G-code generation
 - Cost calculation → Result display
- **Due:** End of Sprint 2 (Feb 28)

7.2 Sequence Diagrams

- **Required Diagrams:**
 1. File upload sequence
 2. Slicing sequence (frontend → backend → CURA → database)
 3. Cost estimation sequence
 4. Authentication sequence
- **Format:** Mermaid.js in Markdown files
- **Due:** End of Sprint 2 (Feb 28)

7.3 Use Case Diagram

- **Already completed in SRS document**
- **Update if new use cases discovered**

8 Known Issues & Risks

8.1 Current Issues

- **[3DP-BUG-1]** PostgreSQL permission errors on some Ubuntu systems
 - Workaround: Use sudo for docker-compose commands
 - Fix planned: Update Docker group permissions in setup docs
- **[3DP-BUG-2]** Three.js bundle size concerns (3.2MB)
 - Impact: Slow initial page load
 - Fix planned: Code splitting, lazy load STLLoader

8.2 Risks for Sprint 2

- **Risk:** CuraEngine integration complexity unknown
 - Mitigation: Allocate 13 story points, start early in sprint
 - Fallback: Use simpler slicing library if CuraEngine blocks progress
- **Risk:** Team member availability (exams, other courses)

- Mitigation: Front-load critical work, flexible task reassignment
- **Risk:** Database authentication issues in production
 - Mitigation: Test with different PostgreSQL auth methods
 - Document setup steps clearly in README

9 Individual Contribution Tracking

9.1 Sprint 1 Contributions (GitHub Stats)

- **Abhiroop:** 8 commits, 2 PRs merged (Swagger docs, database models)
- **Mehrish:** 12 commits, 3 PRs merged (upload API, file validation, tests)
- **Ruschita:** 6 commits, 2 PRs merged (database schema, CI setup)
- **Sai Hasini:** 5 commits, 1 PR merged (frontend project setup)
- **Ronith:** 7 commits, 2 PRs merged (Docker Compose, initial Three.js setup)

9.2 Visibility & Accountability

- GitHub Insights dashboard reviewed weekly
- Each member must merge at least 2 PRs per sprint
- Quality over quantity: Focus on complete, tested features
- Team lead tracks progress and flags concerns early

10 Release Process (Detailed)

10.1 Creating a Release

1. **Code Freeze:** All features merged to dev by Wednesday 6 PM
2. **Create Release Branch:**

```
git checkout dev
git pull origin dev
git checkout -b release/v1.0.0
git push origin release/v1.0.0
```

3. **QA Testing:** Test all features on release branch
 - Run full test suite (unit + integration + E2E)
 - Manual testing checklist (upload, slice, view, estimate)
 - Browser compatibility check (Chrome, Firefox, Safari)
4. **Bug Fixes:** Fix P0/P1 bugs in release branch
 - Commit directly to release branch (no PR needed for urgent fixes)
 - Document all fixes in release notes
5. **Merge to Main:**


```
git checkout main
git merge release/v1.0.0
git push origin main
```

6. Create Git Tag:

```
git tag -a v1.0.0 -m "Release 1.0.0 - MVP Features"
git push origin v1.0.0
```

7. Merge Back to Dev:

```
git checkout dev
git merge main
git push origin dev
```

8. Generate Release Notes: See template below

9. GitHub Release: Create release on GitHub with tag, notes, and build artifacts

10.2 Release Notes Template

Release v1.0.0 - February 28, 2026

New Features

- [3DP-1] File upload API with STL/STEP validation
- [3DP-10] CURA slicing engine integration
- [3DP-15] Interactive 3D model viewer (Three.js)
- [3DP-20] Slicing parameter configuration form
- [3DP-25] Cost estimation with detailed breakdown

Bug Fixes

- [3DP-BUG-1] Fixed PostgreSQL permission errors
- [3DP-BUG-2] Optimized Three.js bundle size

Known Issues

- Large models (>50MB) may take longer to render
- G-code layer preview limited to 1000 layers max

Testing Results

- 45 unit tests passed
- 12 integration tests passed
- E2E workflow: PASSED
- Manual QA: PASSED

Deployment Notes

- Requires Docker 20.10+
- PostgreSQL 15+ (included in docker-compose)
- Node.js 18+ for frontend build

Breaking Changes

None (initial release)

Contributors

Team 8: Abhiroop, Mehrish, Ruschita, Sai Hasini, Ronith

10.3 Hotfix Process

- **When needed:** Critical bug found in production (main branch)
- **Process:**
 1. Create hotfix branch from main: `hotfix/v1.0.1`
 2. Fix bug and test thoroughly
 3. Merge to main and tag as v1.0.1
 4. Merge back to dev
 5. Update release notes
- **Priority:** P0 bugs only, fix within 24 hours

11 Action Items Summary

Owner	Task	Due Date
Abhiroop	Setup CuraEngine Docker container	Feb 15
Abhiroop	Implement /api/slice endpoint	Feb 18
Mehrish	Implement cost calculation engine	Feb 19
Ronith	Complete Three.js viewer with OrbitControls	Feb 17
Sai Hasini	Build slicing parameters form UI	Feb 18
Ruschita	Create control flow & sequence diagrams	Feb 20
All	Test end-to-end workflow (upload → slice → view → estimate)	Feb 25
Team Lead	Prepare Release 1 demo presentation	Feb 27
Team Lead	Generate release notes and create tag	Feb 28

Minutes prepared by: Team 8**Date:** February 13, 2026